

第 2 周：Mirror Descent 与 AdaGrad 的 Regret 分析

上海财经大学 · 计算机机械社

2025-2026 学年 (V2, 2026 年 7 月 12 日)

本周核心思想

「选优化器就是选几何。」

不同的 Bregman divergence 对应不同的几何结构，
Mirror Descent 通过在「镜像空间」中做梯度下降来利用这一几何，
而 AdaGrad 通过在线的对角预处理矩阵自动适应数据的几何。

1 Bregman Divergence

1.1 定义

定义 1.1 (Bregman Divergence). 设 $\psi: \mathcal{X} \rightarrow \mathbb{R}$ 是定义在闭凸集 $\mathcal{X} \subseteq \mathbb{R}^d$ 上的一个严格凸且连续可微的函数。 ψ 诱导的 **Bregman 散度** (Bregman Divergence) 定义为

$$D_\psi(x, y) = \psi(x) - \psi(y) - \langle \nabla \psi(y), x - y \rangle. \quad (1)$$

直观解释： $D_\psi(x, y)$ 度量的是 $\psi(x)$ 与其在点 y 处的一阶泰勒展开 $\psi(y) + \langle \nabla \psi(y), x - y \rangle$ 之间的差距。这正是在给定线性近似下，用 ψ 来度量点 x 与点 y 之间「距离」的一种方式。

Bregman 散度的几何解释

在点 y 处对 ψ 做一阶泰勒展开，得到支撑超平面。 $D_\psi(x, y)$ 就是 $\psi(x)$ 与该超平面在 x 处的垂直距离。由于 ψ 是凸的，该距离始终非负。

1.2 基本性质

1. 非负性： $D_\psi(x, y) \geq 0$ ，且当 ψ 严格凸时 $D_\psi(x, y) = 0 \iff x = y$ 。

2. 非对称性：一般地 $D_\psi(x, y) \neq D_\psi(y, x)$ ——因此它是一种「散度」而非「距离」。
3. 关于 x 的凸性： ψ 的凸性保证了 $D_\psi(\cdot, y)$ 关于第一个变量是凸的。
4. 线性变换下的等变性：若 $\tilde{\psi}(x) = \psi(Ax)$ ，则 $D_{\tilde{\psi}}(x, y) = D_\psi(Ax, Ay)$ 。

1.3 常见例子

名称	$\psi(x)$	$D_\psi(x, y)$
平方欧氏距离	$\frac{1}{2}\ x\ _2^2$	$\frac{1}{2}\ x - y\ _2^2$
Mahalanobis 距离	$\frac{1}{2}x^\top Hx$ ($H \succ 0$)	$\frac{1}{2}(x - y)^\top H(x - y)$
KL 散度	$\sum_i x_i \log x_i$ (单纯形上)	$\sum_i x_i \log \frac{x_i}{y_i}$
Itakura-Saito 距离	$-\sum_i \log x_i$	$\sum_i \left(\frac{x_i}{y_i} - \log \frac{x_i}{y_i} - 1\right)$

关键洞察：不同的 ψ 定义不同的「近邻结构」——即哪些点被认为是「近」的。这正是 Mirror Descent 可以适应不同几何结构的根本原因。

1.4 三点恒等式 (Three-Point Identity)

引理 1.1 (三点恒等式). 设 ψ 为严格凸且连续可微，则对任意 $x, y, z \in \mathcal{X}$ ，有

$$D_\psi(x, y) - D_\psi(x, z) + D_\psi(y, z) = \langle \nabla \psi(z) - \nabla \psi(y), x - y \rangle. \quad (2)$$

证明. 展开左边：

$$\begin{aligned}
& [\psi(x) - \psi(y) - \langle \nabla \psi(y), x - y \rangle] - [\psi(x) - \psi(z) - \langle \nabla \psi(z), x - z \rangle] \\
& \quad + [\psi(y) - \psi(z) - \langle \nabla \psi(z), y - z \rangle] \\
&= \psi(x) - \psi(y) - \langle \nabla \psi(y), x - y \rangle - \psi(x) + \psi(z) + \langle \nabla \psi(z), x - z \rangle \\
& \quad + \psi(y) - \psi(z) - \langle \nabla \psi(z), y - z \rangle \\
&= \langle \nabla \psi(z), x - z \rangle - \langle \nabla \psi(y), x - y \rangle - \langle \nabla \psi(z), y - z \rangle \\
&= \langle \nabla \psi(z), x - y \rangle - \langle \nabla \psi(y), x - y \rangle \\
&= \langle \nabla \psi(z) - \nabla \psi(y), x - y \rangle.
\end{aligned}$$

□

为什么重要？ 三点恒等式是 Mirror Descent regret 分析的核心代数工具。它将 Bregman 散度的变化与梯度差的线性内积联系起来，从而将「散度」语言翻译为「梯度」语言，使我们能够对 regret 进行 telescoping（望远镜求和）。

2 Mirror Descent 框架

2.1 算法定义

在传统的梯度下降中，我们在原始空间更新参数：

$$\theta_{t+1} = \theta_t - \eta g_t.$$

这等价于求解近端问题：

$$\theta_{t+1} = \operatorname{argmin}_{\theta \in \mathcal{X}} \left\{ \eta \langle g_t, \theta \rangle + \frac{1}{2} \|\theta - \theta_t\|_2^2 \right\}.$$

Mirror Descent (MD) 将欧氏距离 $\frac{1}{2} \|\theta - \theta_t\|_2^2$ 替换为一般的 Bregman 散度 $D_\psi(\theta, \theta_t)$ ：

定义 2.1 (Mirror Descent 更新规则). 给定学习率 $\eta > 0$ 和 Bregman 势函数 ψ ,

$$\theta_{t+1} = \operatorname{argmin}_{\theta \in \mathcal{X}} \left\{ \eta \langle g_t, \theta \rangle + D_\psi(\theta, \theta_t) \right\}. \quad (3)$$

2.2 从 MD 到预条件梯度下降

定理 2.1 (MD 退化为预条件 GD). 取 $\psi(\theta) = \frac{1}{2} \theta^\top H \theta$, 其中 $H \succ 0$ (对称正定矩阵), 则 MD 更新退化为

$$\theta_{t+1} = \theta_t - \eta H^{-1} g_t. \quad (4)$$

证明. 代入 $\psi(\theta) = \frac{1}{2} \theta^\top H \theta$, 有

$$D_\psi(\theta, \theta_t) = \frac{1}{2} \theta^\top H \theta - \frac{1}{2} \theta_t^\top H \theta_t - (H \theta_t)^\top (\theta - \theta_t) = \frac{1}{2} (\theta - \theta_t)^\top H (\theta - \theta_t).$$

MD 子问题为

$$\min_{\theta \in \mathcal{X}} \left\{ \eta \langle g_t, \theta \rangle + \frac{1}{2} (\theta - \theta_t)^\top H (\theta - \theta_t) \right\}.$$

若 $\mathcal{X} = \mathbb{R}^d$ (无约束), 则求导得最优条件：

$$\eta g_t + H(\theta - \theta_t) = 0 \implies \theta = \theta_t - \eta H^{-1} g_t.$$

□

解读： H 对梯度做了「预条件」(preconditioning) —— 在 H 的特征值较大的方向 (即 ψ 在该方向更陡峭), 步长被缩小; 在 H 的特征值较小的方向, 步长被放大。通过明智地选择 H , 我们可以做到适应数据几何的优化。

「镜像」的直觉

MD 将参数 θ_t 通过**镜像映射** $\nabla\psi$ 映射到对偶空间: $y_t = \nabla\psi(\theta_t)$ 。在对偶空间中做标准的梯度下降: $y_{t+1} = y_t - \eta g_t$ 。然后再通过 $(\nabla\psi)^{-1}$ 拉回原始空间: $\theta_{t+1} = (\nabla\psi)^{-1}(y_{t+1})$ 。

这就是「镜像」(Mirror) 的由来: 原始空间的非欧更新, 可以被理解为在对偶空间中的欧氏更新。

2.3 镜像空间视角

关键的几何洞察: ψ 的选择决定了什么是一个「好的」局部距离度量。例如:

- $\psi(x) = \frac{1}{2}\|x\|_2^2 \rightarrow$ 欧氏几何, 适合稠密梯度场景;
- $\psi(x) = \sum_i x_i \log x_i$ (负熵) $\rightarrow$ KL 几何, 适合单纯形上的概率分布更新 (如 EG 算法);
- $\psi(\theta) = \frac{1}{2}\theta^\top H_t \theta$ (H_t 随时间变化) $\rightarrow$ AdaGrad 的思想雏形。

3 在线凸优化 (OCO) 与 Regret**3.1 OCO 基本设定**

在在线凸优化中, 学习器和环境进行 T 轮的交互:

每一轮 $t = 1, 2, \dots, T$:

1. 学习器选择一个决策 $\theta_t \in \mathcal{X}$ (凸集)。
2. 环境 (具有完全信息的对手) 揭示一个凸损失函数 $\ell_t(\cdot)$ 。
3. 学习器承受损失 $\ell_t(\theta_t)$ 并观察 (子) 梯度 $g_t \in \partial\ell_t(\theta_t)$ 。

我们希望学习器的累计损失不比事后最优的固定决策差太多。

定义 3.1 (Regret (遗憾)). 学习器在 T 轮后的 regret 定义为

$$\text{Regret}_T = \sum_{t=1}^T \ell_t(\theta_t) - \min_{\theta \in \mathcal{X}} \sum_{t=1}^T \ell_t(\theta). \quad (5)$$

- Regret_T 是实际损失与事后最优固定决策的损失之差。
- 目标: 设计算法使得 $\text{Regret}_T = o(T)$, 即平均 regret 趋近于零。
- 更理想的目标: $\text{Regret}_T = O(\sqrt{T})$ 。

3.2 Mirror Descent 的 Regret Bound (通用版)

定理 3.1 (MD 通用 Regret Bound). 设 ψ 在 $\mathcal{X}$ 上是 ρ -强凸的 (关于范数 $\|\cdot\|$), 学习率 $\eta > 0$ 。则 *Mirror Descent* 的 regret 满足

$$\text{Regret}_T \leq \frac{1}{\eta} D_\psi(\theta^*, \theta_1) + \frac{\eta}{2\rho} \sum_{t=1}^T \|g_t\|_*^2, \quad (6)$$

其中 $\|\cdot\|_*$ 是 $\|\cdot\|$ 的对偶范数, $\theta^* = \operatorname{argmin}_{\theta \in \mathcal{X}} \sum_t \ell_t(\theta)$ 。

证明 (骨架). 令 θ^* 为最优固定决策。由式 (3) 的最优条件:

$$\langle \eta g_t + \nabla \psi(\theta_{t+1}) - \nabla \psi(\theta_t), \theta^* - \theta_{t+1} \rangle \geq 0.$$

利用三点恒等式 (引理 1.1) 消去 $\nabla \psi$, 得到

$$\eta \langle g_t, \theta_t - \theta^* \rangle \leq D_\psi(\theta^*, \theta_t) - D_\psi(\theta^*, \theta_{t+1}) + \frac{\eta}{2\rho} \|g_t\|_*^2.$$

对 $t = 1$ 到 T 求和, 中间项 telescoping:

$$\sum_{t=1}^T [D_\psi(\theta^*, \theta_t) - D_\psi(\theta^*, \theta_{t+1})] = D_\psi(\theta^*, \theta_1) - D_\psi(\theta^*, \theta_{T+1}) \leq D_\psi(\theta^*, \theta_1).$$

最后, 由损失函数的凸性 $\ell_t(\theta_t) - \ell_t(\theta^*) \leq \langle g_t, \theta_t - \theta^* \rangle$, 即得结论。□

该 bound 告诉我们什么?

- 第一项 $D_\psi(\theta^*, \theta_1)/\eta$ 度量初始化误差——与「最佳固定决策离初始点有多远」有关。
- 第二项 $(\eta/2\rho) \sum_t \|g_t\|_*^2$ 累积了梯度信息——反映了损失函数的「难度」。
- 最优学习率 $\eta = \Theta(1/\sqrt{T})$ 会给出 $\text{Regret}_T = O(\sqrt{T})$ 。

4 AdaGrad

4.1 动机: 为什么需要自适应方法?

回顾 SGD 的 regret bound (取 $\psi(x) = \frac{1}{2}\|x\|_2^2$, $\rho = 1$, $\|\cdot\|_* = \|\cdot\|_2$):

$$\text{Regret}_T^{\text{SGD}} \leq \frac{D}{2\eta} + \frac{\eta}{2} \sum_{t=1}^T \|g_t\|_2^2.$$

取最优 η 后:

$$\text{Regret}_T^{\text{SGD}} = O\left(D \sqrt{\sum_{t=1}^T \|g_t\|_2^2}\right).$$

问题在哪? 当梯度是稠密的 (即每个坐标在每个时刻都有非零梯度), $\sum_t \|g_t\|_2^2 = \Theta(dT)$, 于是 $\text{Regret}_T^{\text{SGD}} = O(\sqrt{dT})$ 。

但实际中梯度往往是稀疏的——例如只有 5% 的坐标在每轮被激活。SGD 的 bound 并不能利用这种稀疏性, 因为它对每个坐标施加了相同的有效学习率。 $\|g_t\|_2^2$ 这个聚合指标掩盖了「哪些特征经常出现、哪些很少出现」的结构信息。

AdaGrad 的核心思想: 在每一轮, 不仅要知道梯度的大小, 还要知道每个坐标过去被更新的频率, 从而对活跃度高的特征使用较小的学习率, 对活跃度低的特征使用较大的学习率。

4.2 对角 AdaGrad 算法

算法 1: 对角 AdaGrad (Diagonal AdaGrad)

输入: 初始参数 $\theta_1 \in \mathbb{R}^d$, 常数 $\varepsilon > 0$ 。

初始化: $s_0 = \mathbf{0} \in \mathbb{R}^d$ (或 $s_{0,i} = 0, \forall i$)。

for $t = 1, 2, \dots, T$ **do**

1. 做预测 θ_t 。

2. 接收损失函数 $\ell_t(\cdot)$ 并计算子梯度 $g_t \in \partial \ell_t(\theta_t)$ 。

3. 更新累积平方梯度: $s_{t,i} = s_{t-1,i} + g_{t,i}^2, \quad \forall i = 1, \dots, d$ 。

4. 参数更新 (每坐标自适应学习率): $\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{s_{t,i} + \varepsilon}} g_{t,i}, \quad \forall i$ 。

end for

关键结构:

- 每个坐标 i 拥有独立的学习率 $\eta_{t,i} = \eta / \sqrt{s_{t,i} + \varepsilon}$ 。
- $s_{t,i} = \sum_{k=1}^t g_{k,i}^2$ 是该坐标历史上所有梯度平方的累积和。
- 频繁更新的坐标 ($s_{t,i}$ 大) $\rightarrow$ 学习率小; 稀疏更新的坐标 ($s_{t,i}$ 小) $\rightarrow$ 学习率大。
- ε 用于避免除零。

4.3 AdaGrad 的 Regret Bound 推导

定理 4.1 (对角 AdaGrad 的 Regret Bound). 设 ℓ_t 为凸函数, $\|g_t\|_\infty \leq G_\infty$, $D_\infty = \max_{\theta \in \mathcal{X}} \|\theta - \theta_1\|_\infty$ 。则对角 AdaGrad 满足

$$\text{Regret}_T \leq \sqrt{2} D_\infty \sum_{i=1}^d \sqrt{\sum_{t=1}^T g_{t,i}^2}. \quad (7)$$

推导步骤

第 1 步：MD 视角下的 AdaGrad。

考虑时变的 Bregman 势函数：

$$\psi_t(\theta) = \frac{1}{2}\theta^\top H_t \theta, \quad \text{其中} \quad H_t = \frac{1}{\eta} \text{diag}(\sqrt{s_{t,1} + \varepsilon}, \dots, \sqrt{s_{t,d} + \varepsilon}).$$

此时第 t 步的 MD 更新为

$$\theta_{t+1} = \underset{\theta}{\operatorname{argmin}} \left\{ \eta \langle g_t, \theta \rangle + D_{\psi_t}(\theta, \theta_t) \right\} = \theta_t - \eta H_t^{-1} g_t,$$

这正是 AdaGrad 的更新规则。

第 2 步：单步 regret 分解。

考虑损失函数为线性函数 $\ell_t(\theta) = \langle g_t, \theta \rangle$ 的情形（一般凸情形用一阶 Taylor 展开，结论一致）。我们有：

$$\langle g_t, \theta_t - \theta^* \rangle \leq D_{\psi_t}(\theta^*, \theta_t) - D_{\psi_t}(\theta^*, \theta_{t+1}) + \frac{\eta}{2} \|g_t\|_{H_t^{-1}}^2 \quad (8)$$

$$= D_{\psi_t}(\theta^*, \theta_t) - D_{\psi_t}(\theta^*, \theta_{t+1}) + \frac{\eta}{2} \sum_{i=1}^d \frac{g_{t,i}^2}{\sqrt{s_{t,i} + \varepsilon}}. \quad (9)$$

这使用了 $H_t^{-1} = \eta \cdot \text{diag}(1/\sqrt{s_{t,1} + \varepsilon}, \dots, 1/\sqrt{s_{t,d} + \varepsilon})$ 。

第 3 步：Telescoping 的非平凡之处。

由于 ψ_t 随时间变化， $D_{\psi_t}(\theta^*, \theta_t)$ 和 $D_{\psi_{t-1}}(\theta^*, \theta_t)$ 不同，telescoping 不能简单地跨步进行。我们需要处理「漂移项」：

$$\begin{aligned} & \sum_{t=1}^T [D_{\psi_t}(\theta^*, \theta_t) - D_{\psi_t}(\theta^*, \theta_{t+1})] \\ &= D_{\psi_1}(\theta^*, \theta_1) - D_{\psi_T}(\theta^*, \theta_{T+1}) + \sum_{t=1}^{T-1} [D_{\psi_{t+1}}(\theta^*, \theta_{t+1}) - D_{\psi_t}(\theta^*, \theta_{t+1})]. \end{aligned}$$

对于 $\psi_t(\theta) = \frac{1}{2\eta} \theta^\top \text{diag}(\sqrt{s_t + \varepsilon}) \theta$ ，漂移项为

$$D_{\psi_{t+1}}(\theta^*, \theta_{t+1}) - D_{\psi_t}(\theta^*, \theta_{t+1}) = \frac{1}{2\eta} \sum_{i=1}^d (\theta_i^* - \theta_{t+1,i})^2 (\sqrt{s_{t+1,i} + \varepsilon} - \sqrt{s_{t,i} + \varepsilon}).$$

第 4 步：对 T 步求和的最终 bound。

综合第 2 步和第 3 步，经过细致的代数操作（详见 Duchi et al. 2011 附录 A），可以证明：

$$\sum_{t=1}^T \langle g_t, \theta_t - \theta^* \rangle \leq \frac{1}{2\eta} \sum_{i=1}^d (\theta_i^* - \theta_{1,i})^2 \sqrt{s_{T,i} + \varepsilon} + \frac{\eta}{2} \sum_{t=1}^T \sum_{i=1}^d \frac{g_{t,i}^2}{\sqrt{s_{t,i} + \varepsilon}}.$$

取 η 最优, 并使用不等式

$$\sum_{t=1}^T \frac{g_{t,i}^2}{\sqrt{s_{t,i}} + \varepsilon} \leq 2\sqrt{s_{T,i}} + \varepsilon,$$

(这由 $\sum_t a_t / \sqrt{\sum_{k \leq t} a_k} \leq 2\sqrt{\sum_t a_t}$ 这一标准引理保证, 其中 $a_t = g_{t,i}^2$), 我们得到:

$$\text{Regret}_T \leq \sqrt{2} D_\infty \sum_{i=1}^d \sqrt{\sum_{t=1}^T g_{t,i}^2}.$$

4.4 为什么在稀疏梯度下远优于 SGD?

将 AdaGrad 的 bound 与 SGD 的 bound 做对比。

场景: 假设每个样本只激活 $k \ll d$ 个坐标 (例如 $k = 0.05d$), 且非零梯度的值约为 ± 1 。

• **SGD Bound:** $\text{Regret}_T^{\text{SGD}} = O\left(D_\infty \sqrt{\sum_{t=1}^T \sum_{i=1}^d g_{t,i}^2}\right) = O(D_\infty \sqrt{kT}).$

• **AdaGrad Bound:** $\text{Regret}_T^{\text{AdaGrad}} = O\left(D_\infty \sum_{i=1}^d \sqrt{\sum_{t=1}^T g_{t,i}^2}\right) = O\left(D_\infty \cdot d \cdot \sqrt{\frac{kT}{d}}\right) = O(D_\infty \sqrt{dkT}).$

等等——乍一看 AdaGrad 的 bound 似乎更差 (因子 $\sqrt{dk}$ vs $\sqrt{k}$)。

关键在于 D_∞ 的定义不同! 上面的比较忽略了 D_∞ 的量纲差异:

- SGD 的 D_∞ 是在 ℓ_2 意义下度量的, 即 $D_\infty = \max \|\theta - \theta_1\|_2$ 。
- AdaGrad 的 D_∞ 是在 ℓ_∞ 意义下度量的, 即 $D_\infty = \max \|\theta - \theta_1\|_\infty$ 。

更精确的比较应当对 bound 取最优 trade-off。现在我们看最重要的洞察:

稀疏梯度的优势:

考虑每个坐标 i 在整个 T 轮中被激活的总次数为 $\tau_i \ll T$ 。则

$$\sum_{i=1}^d \sqrt{\sum_{t=1}^T g_{t,i}^2} \approx \sum_{i=1}^d \sqrt{\tau_i}.$$

而 SGD 的 bound 中,

$$\sqrt{\sum_{t=1}^T \|g_t\|_2^2} = \sqrt{\sum_{t=1}^T \sum_{i=1}^d g_{t,i}^2} \approx \sqrt{kT}.$$

当 τ_i 极度不均匀时 ($k \ll d$ 且某些特征主导), 前者可以远小于后者。例如, 若只有 $O(1)$ 个特征被频繁激活, 其余特征的 $\tau_i = O(1)$, 则

$$\sum_i \sqrt{\tau_i} = O(1 \cdot \sqrt{T} + d \cdot 1) = O(\sqrt{T} + d).$$

相对地, SGD bound 总是 $O(\sqrt{dT})$ ——即使只有很少的特征是活跃的。

因此, **AdaGrad** 在极端稀疏的数据 (如文本分类中的 **bag-of-words** 特征) 上能够实现远优于 **SGD** 的实际收敛。

4.5 从「Regret Bound」到「实际收敛」的桥梁

在 OCO 设定下, $\text{Regret}_T/T \rightarrow 0$ 意味着平均损失趋近于最优固定决策的损失。对于随机优化 (随机抽取 i.i.d. 数据), 如果使用在线到批次的转换 (online-to-batch conversion), AdaGrad 的 regret bound 直接给出:

$$\mathbb{E}[F(\bar{\theta}_T)] - F(\theta^*) = O\left(\frac{1}{T} \sum_{i=1}^d \mathbb{E}\left[\sqrt{\sum_{t=1}^T g_{t,i}^2}\right]\right),$$

其中 $\bar{\theta}_T = \frac{1}{T} \sum_{t=1}^T \theta_t$ 。这使得 regret 分析成为算法性能保证的坚实基础。

5 FTRL (Follow-The-Regularized-Leader)

5.1 基本概念

Mirror Descent 不是唯一的在线学习框架。另一个核心框架是 **FTRL (Follow-The-Regularized-Leader)**。

定义 5.1 (FTRL 更新规则). 在第 t 轮, FTRL 选择

$$\theta_{t+1} = \operatorname{argmin}_{\theta \in \mathcal{X}} \left\{ \sum_{k=1}^t \langle g_k, \theta \rangle + \frac{1}{\eta} R(\theta) \right\}, \quad (10)$$

其中 $R(\theta)$ 是一个正则化函数 (通常是强凸的)。

直觉: FTRL 直接选择那个在过去所有时刻的线性损失之和下表现最好的决策, 同时用 $R(\theta)$ 做正则化以防止过拟合到有限的历史。

5.2 MD 与 FTRL 的关系

- MD 是一种**增量式**方法——每步在上一点的基础上做局部更新。
- FTRL 是一种**批量式**方法——每步求解一个全局优化问题。
- 在特定条件下 (线性损失 + 固定正则化器), MD 和 FTRL 是**等价的**。更精确地说, 若 $R(\theta) = \psi(\theta) - \psi(\theta_1)$ (在合适的边界条件下), 则 MD 产生的序列与 FTRL 产生的序列一致。
- 对于时变的正则化器 (如 AdaGrad 中的 H_t), FTRL 的推广版 FTRL-Proximal 可以与 AdaGrad 取得类似的 regret bound。

5.3 为什么引入 FTRL?

1. **统一视角:** AdaGrad 可以从「时变正则化器的 FTRL」角度推导出来。原始 AdaGrad 论文 (Duchi et al. 2011) 正是使用了 FTRL 框架。
2. **稀疏性:** FTRL 配合 ℓ_1 正则化 (尤其是 FTRL-Proximal) 可以产生稀疏解, 这对大规模在线学习 (如 Google 的广告点击率预测) 至关重要。
3. **理论优雅性:** FTRL 的 regret 分析往往比 MD 更加直接——它不需要三点恒等式, 而是直接使用凸函数的定义和最优条件。

6 工程实践

6.1 实现对角 AdaGrad

以下 Python 实现展示了对角 AdaGrad 的核心逻辑:

Listing 1: 对角 AdaGrad 的 NumPy 实现

```
1 import numpy as np
2
3 class DiagonalAdaGrad:
4     """对角 AdaGrad 优化器。
5
6     参数
7     -----
8     lr : float
```

```

9         全局学习率 eta。
10    eps : float
11        平滑项, 避免除零 (默认 1e-8)。
12    """
13    def __init__(self, lr=0.1, eps=1e-8):
14        self.lr = lr
15        self.eps = eps
16        self.s = None                                # 累积平方梯度
17
18    def update(self, theta, grad):
19        """对参数执行一次更新的原地操作。
20
21        参数
22        -----
23        theta : ndarray, shape (d,)
24            当前参数向量。
25        grad : ndarray, shape (d,)
26            当前批次的梯度。
27        """
28        if self.s is None:
29            self.s = np.zeros_like(theta)
30
31        # 累积平方梯度:  $s_{\{t,i\}} = s_{\{t-1,i\}} + g_{\{t,i\}}^2$ 
32        self.s += grad ** 2
33
34        # 自适应更新:  $\theta_{\{t+1,i\}} = \theta_{\{t,i\}} - \eta / \sqrt{s_{\{t,i\}} + \epsilon} * g_{\{t,i\}}$ 
35        theta -= self.lr / (np.sqrt(self.s) + self.eps) * grad

```

6.2 构造稀疏梯度场景

Listing 2: 稀疏梯度场景的构造

```

1 def generate_sparse_gradient_data(d, sparsity=0.05, n_samples
2   =10000):
3     """
4     生成稀疏梯度场景的模拟数据。
5
6     每个样本仅在随机选择的 5% 坐标上拥有非零梯度,
    以此模拟文本分类、推荐系统中常见的稀疏特征。

```

```

7
8     参数
9     -----
10    d : int
11        数据维度。
12    sparsity : float
13        每个样本中非零特征的比例。
14    n_samples : int
15        样本数量。
16
17    返回
18    -----
19    X : ndarray, shape (n_samples, d)
20        特征矩阵。
21    y : ndarray, shape (n_samples,)
22        标签。
23    theta_star : ndarray, shape (d,)
24        真实的「最优」参数（用于生成标签）。
25    """
26    np.random.seed(42)
27    theta_star = np.random.randn(d)          # 真实参数
28
29    X = np.zeros((n_samples, d))
30    for i in range(n_samples):
31        # 随机选择 5% 的坐标作为活跃特征
32        active = np.random.choice(d, size=int(d * sparsity),
33                                   replace=False)
34        X[i, active] = np.random.randn(len(active))
35
36    # 生成带噪声的标签
37    y = X @ theta_star + 0.1 * np.random.randn(n_samples)
38    return X, y, theta_star

```

6.3 比较 SGD vs AdaGrad

Listing 3: SGD vs AdaGrad 在稀疏场景的收敛比较

```

1 import matplotlib.pyplot as plt
2
3 def compare_optimizers_sparse(d=1000, sparsity=0.05,

```

```
4         n_samples=5000, epochs=20):
5     """在稀疏梯度场景下比较 SGD 和 AdaGrad."""
6     X, y, theta_star = generate_sparse_gradient_data(
7         d, sparsity, n_samples
8     )
9
10    # 初始化
11    theta_sgd = np.zeros(d)
12    theta_adagrad = np.zeros(d)
13    adagrad = DiagonalAdaGrad(lr=0.1, eps=1e-8)
14
15    loss_sgd, loss_adagrad = [], []
16
17    for epoch in range(epochs):
18        perm = np.random.permutation(n_samples)
19        total_loss_sgd = 0.0
20        total_loss_adagrad = 0.0
21
22        for idx in perm:
23            xi, yi = X[idx], y[idx]
24
25            # --- SGD 更新 ---
26            pred_sgd = np.dot(xi, theta_sgd)
27            grad_sgd = 2 * (pred_sgd - yi) * xi
28            theta_sgd -= 0.01 * grad_sgd      # 固定学习率
29            total_loss_sgd += (pred_sgd - yi) ** 2
30
31            # --- AdaGrad 更新 ---
32            pred_ag = np.dot(xi, theta_adagrad)
33            grad_ag = 2 * (pred_ag - yi) * xi
34            adagrad.update(theta_adagrad, grad_ag)
35            total_loss_adag += (pred_ag - yi) ** 2
36
37        loss_sgd.append(np.sqrt(total_loss_sgd / n_samples))
38        loss_adagrad.append(np.sqrt(total_loss_adag / n_samples))
39        print(f"Epoch {epoch+1:2d}: "
40              f"SGD RMSE={loss_sgd[-1]:.4f}, "
41              f"AdaGrad RMSE={loss_adagrad[-1]:.4f}")
42
43    # 绘图
```

```

44 plt.figure(figsize=(8, 5))
45 plt.plot(loss_sgd, 'o-', label='SGD (lr=0.01)', alpha=0.8)
46 plt.plot(loss_adagrad, 's-', label='AdaGrad (lr=0.1)', alpha
    =0.8)
47 plt.xlabel('Epoch')
48 plt.ylabel('RMSE')
49 plt.title(f'稀疏梯度场景 (d={d}, sparsity={sparsity:.0%})')
50 plt.legend()
51 plt.grid(True, alpha=0.3)
52 plt.tight_layout()
53 plt.savefig('sgd_vs_adagrad_sparse.pdf', dpi=150)
54 plt.show()
55
56 if __name__ == '__main__':
57     compare_optimizers_sparse()

```

预期结果：在稀疏梯度场景 ($d = 1000$, 仅 5% 特征活跃) 中, AdaGrad 的收敛速度应明显快于 SGD。SGD 对所有坐标使用相同的学习率, 导致在稀疏特征上的有效步长过小; AdaGrad 为每个坐标自适应地调整学习率, 使得稀疏坐标获得更大的步长。

6.4 可视化每坐标的有效学习率

Listing 4: 可视化每坐标有效学习率随时间的变化

```

1 def visualize_learning_rates(d=100, sparsity=0.05,
2                               T=500):
3     """
4     可视化 AdaGrad 中每个坐标的有效学习率变化。
5
6     在稀疏场景下, 可以清楚地看到活跃坐标与非活跃坐标
7     的学习率衰减速度的巨大差异。
8     """
9     np.random.seed(123)
10
11     # 记录每个坐标的有效学习率
12     lr_history = np.zeros((T, d))
13     s = np.zeros(d)
14     eta = 0.1
15     eps = 1e-8
16

```

```
17     for t in range(T):
18         # 模拟稀疏梯度: 每轮随机激活 5% 坐标
19         grad = np.zeros(d)
20         active = np.random.choice(
21             d, size=int(d * sparsity), replace=False
22         )
23         grad[active] = np.random.randn(len(active))
24
25         s += grad ** 2
26         eff_lr = eta / (np.sqrt(s) + eps)
27         lr_history[t] = eff_lr
28
29     # 绘图
30     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
31
32     # 左图: 一些坐标的有效学习率轨迹
33     ax = axes[0]
34     # 选择活跃度不同的几个坐标
35     coord_active = active[0]      # 持续活跃
36     coord_rare = (active[0] + 50) % d # 可能较不活跃
37     coord_inactive = d - 1        # 几乎不活跃
38
39     ax.semilogy(lr_history[:, coord_active], label=f'坐标 {
40         coord_active} (活跃)')
41     ax.semilogy(lr_history[:, coord_rare], label=f'坐标 {
42         coord_rare} (中等)')
43     ax.semilogy(lr_history[:, coord_inactive], label=f'坐标 {
44         coord_inactive} (不活跃)')
45     ax.set_xlabel('轮数 t')
46     ax.set_ylabel('有效学习率 (log scale)')
47     ax.set_title('各坐标的有效学习率随时间衰减')
48     ax.legend(fontsize=8)
49     ax.grid(True, alpha=0.3)
50
51     # 右图: 最终时刻各坐标学习率的直方图
52     ax = axes[1]
53     ax.hist(lr_history[-1], bins=30, edgecolor='black',
54             alpha=0.7)
55     ax.set_xlabel('有效学习率')
56     ax.set_ylabel('坐标数')
```

```
54 ax.set_title(f'T={T} 时有效学习率的分布')
55 ax.grid(True, alpha=0.3)
56
57 plt.tight_layout()
58 plt.savefig('adagrad_lr_visualization.pdf', dpi=150)
59 plt.show()
60
61 if __name__ == '__main__':
62     visualize_learning_rates()
```

关键观察:

- 频繁更新的坐标的有效学习率迅速衰减（在 $\log$ 刻度下呈线性下降趋势）。
- 极少更新的坐标的学习率保持高位——只要它们出现，就能获得一次大幅的更新。
- 最终时刻的有效学习率分布呈长尾状——少量坐标学习率很小（经常被更新），大量坐标学习率接近初始值（很少被更新）。

7 检验标准

完成本周学习后，请确保能够：

1. **Mirror Descent 几何**: 阐明 Mirror Descent 如何通过改变 Bregman divergence 来改变优化几何。能够举出至少两个不同的 ψ 及其对应的几何与适用场景。
2. **退化推导**: 手推当 $\psi(\theta) = \frac{1}{2}\theta^\top H\theta$ 时 MD 如何退化为预条件梯度下降。
3. **AdaGrad Regret Bound 骨架**: 能够写出对角 AdaGrad regret bound $R_T \leq \sqrt{2D_\infty \sum_{i=1}^d \sqrt{\sum_t g_{t,i}^2}}$ 的推导骨架（不要求严格逐行写出所有代数，但要知道每一步用到了什么核心工具）：
 - 单步 regret 分解（Bregman 散度 + 三点恒等式）
 - 漂移项的处理（时变正则化器）
 - 数学引理 $\sum_t a_t / \sqrt{\sum_{k \leq t} a_k} \leq 2\sqrt{\sum_t a_t}$
 - 最优 η 的选取与最终 bound 的合并
4. **稀疏优势**: 能用自己的话解释为什么 AdaGrad 的 regret bound 在稀疏梯度场景下远优于 SGD 的 bound。
5. **FTRL 概念**: 能解释 FTRL 的基本思想，并说明其与 MD 的关系。

8 思考题（选做）

练习 8.1 (Bregman 散度的实验). 取 $\psi_1(x) = \frac{1}{2}x^2$ 和 $\psi_2(x) = x \log x$ (在 $[0, 1]$ 上). 对于 $y = 0.5$, 绘制 $D_{\psi_1}(x, y)$ 与 $D_{\psi_2}(x, y)$ 随 x 的变化曲线. 观察它们在不对称性、在边界附近的行为差异。

练习 8.2 (AdaGrad Bound 的直接检验). 在 $d = 100$ 的场景下, 模拟 $T = 1000$ 轮, 随机生成稀疏梯度. 计算并比较:

$$\sum_{t=1}^T \ell_t(\theta_t^{\text{SGD}}) - \sum_{t=1}^T \ell_t(\theta^*), \quad \sum_{t=1}^T \ell_t(\theta_t^{\text{AdaGrad}}) - \sum_{t=1}^T \ell_t(\theta^*).$$

验证实际 regret 是否逼近理论 bound 所预测的行为。

练习 8.3 (FTRL-Proximal 的实现). 实现 FTRL-Proximal 算法并将其应用于与 AdaGrad 相同的稀疏场景. 比较二者的稀疏性 (解中零元素的比例) 和收敛速度。

参考文献

1. **Duchi, J., Hazan, E., & Singer, Y. (2011).** *Adaptive Subgradient Methods for Online Learning and Stochastic Optimization*. Journal of Machine Learning Research, 12, 2121-2159.
 - 这是 AdaGrad 的原始论文。定理 1-3 分别给出了全矩阵 AdaGrad、对角 AdaGrad 和复合目标下的 regret bound。
2. **Beck, A., & Teboulle, M. (2003).** *Mirror Descent and Nonlinear Projected Subgradient Methods for Convex Optimization*. Operations Research Letters, 31(3), 167-175.
 - Mirror Descent 的经典论文, 系统化了 Bregman divergence 在优化中的应用。
3. **McMahan, H. B. (2011).** *Follow-the-Regularized-Leader and Mirror Descent: Equivalence Theorems and L1 Regularization*. In AISTATS.
 - 系统证明了 MD 与 FTRL 之间的等价关系, 并推广到 FTRL-Proximal。
4. **Hazan, E. (2016).** *Introduction to Online Convex Optimization*. Foundations and Trends in Optimization, 2(3-4), 157-325.
 - 在线凸优化的全面入门教材, 涵盖 MD、FTRL 以及自适应方法。